Workgroup:	Network Working Group
Internet-Draft:	draft-perry-dtn-cpb-00
Published:	29 July 2026
Intended Status:	Experimental
Expires:	30 January 2027
Author:	N. Perry *Independent*

Probabilistic Contact Metadata for DTN Bundle Routing

Abstract

This document defines a generic uncertainty propagation layer for Delay/Disruption Tolerant Networking (DTN) routing: the Contact Probability Block (CPB), a BPv7 extension block that carries probability metadata in-bundle between forwarding nodes. The contribution is the standardized transport mechanism rather than any single routing algorithm; CPB is designed to be consumed and produced by heterogeneous DTN routing protocols (PRoPHET, CGR, MaxProp, RAPID, Spray-and-Wait). Integrating CPB is an optional consumer/producer change at the routing agent; the block does not replace those algorithms.

The specification uses CBOR floating-point encoding in bundle extension blocks, with half-precision (IEEE 754 binary16) recommended for bandwidth efficiency. The extension preserves endpoint identifier immutability, is backwards-compatible with BPv7, and integrates with existing security mechanisms (BPSec). Distinct probability semantics from different routing families are kept separable via an explicit metric-type tag; cross-metric arithmetic is prohibited.

This document is Experimental. The accompanying Configuration 1 simulation compares confidence-blind earliest-arrival CGR to one confidence-weighted consumer of ground-truth confidences of the class CPB is designed to carry (metric-type 1; cost = latency / confidence). On that topology the confidence-weighted consumer improves mean delivery ratio and mean latency while selecting higher path confidence; p95 latency is not improved on this topology and is reported as measured. The contribution of this specification is the standardized BPv7 extension block for carrying such metadata, not a claim that production stacks already consume CPB in-band. Operational deployment questions — robustness to noisy or adversarial estimates, control-theoretic stability under feedback, and interoperability across independent BPv7 implementations — are enumerated as open issues and are not claimed as resolved by this document.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 30 January 2027.

Copyright Notice

Table of Contents

1. Introduction 5
1.1. Problem Statement (IPNSIG SSI Architecture, Section 4.2) 5
1.2. Motivation 6
1.3. Requirements Language 7
1.4. Terminology 7
1.5. Notation Conventions 7
2. Architectural Considerations 7
2.1. Why NOT URI Query Parameters 8
2.1.1. Endpoint Identifier Immutability 8
2.1.2. Router Modification Problem 8
2.2. Solution: Bundle Extension Blocks 9
3. Contact Probability Block 9
3.1. Overview 9
3.2. Block Structure 9
3.2.1. Block Processing Flags 12
3.2.2. Cyclic Redundancy Check (CRC) Handling 13
3.2.3. Hop Count Block Co-Presence 13
3.3. Block Type Code 13
3.3.1. IANA Registration 13

3.3.2. Backwards Compatibility 13
3.4. CBOR Encoding of Probability Values 14
3.4.1. Invalid Float Handling 14
3.4.2. Encoding Details 14
3.4.3. Hex Encoding Table 15
3.5. Metric-Type Semantics and Cross-Metric Comparison 15
3.6. Multiple CPBs in One Bundle 17
3.6.1. Per-Path Matching Rules 17
4. Operational Semantics 18
4.1. Creation 18
4.2. Processing by Intermediate Routers 18
4.3. Usage in Routing Algorithms 19
4.4. Stability Considerations and Feedback Dynamics 19
5. Relationship to Existing DTN Routing Protocols 20
5.1. PRoPHET (RFC 6693) 20
5.2. MaxProp 20
5.3. RAPID 20
5.4. Spray-and-Wait Family 21
5.5. Epidemic and Summary-Vector Protocols 21
5.6. Contact Graph Routing (CGR) 21
5.7. Social and Hybrid Metrics 21
5.8. DTNMA (RFC 9675) 22
5.9. Summary: CPB as Cross-Cutting Metadata 22
6. Backwards Compatibility 22
7. Operational Considerations 22
7.1. When and Where to Emit CPBs 22
7.2. Bundle Store and Memory Sizing 23
7.3. Computational and Storage Overhead on Constrained Nodes 23
7.4. Logging, Monitoring, and OAM 23
7.5. Partial Deployment 23

7.6. Congestion and Storage Pressure 23
7.7. Clock Considerations 24
8. Security Considerations 24
8.1. Threat Model 24
8.2. Trust Model 24
8.2.1. Who is Authorized to Append CPBs 24
8.2.2. Cryptographic Adjudication of Conflicting CPBs 25
8.2.3. Downstream Handling of Upstream BIBs 25
8.2.4. Route Poisoning by Semi-Trusted Relays 25
8.2.5. Inter-Domain Trust 26
8.2.6. Replay Resistance 26
8.3. Integrity Protection 26
8.3.1. Source CPB with Append-Only Intermediates 26
8.3.2. Strip-and-Replace Model 26
8.3.3. Prohibition on BCB Encryption 26
8.4. Mitigations 26
8.5. Privacy Considerations 27
9. IANA Considerations 27
10. Overhead Analysis 27
10.1. Computational and Storage Overhead 28
11. Implementation Status 29
12. Experiment Description 30
12.1. Scope and Hypothesis 30
12.2. Test Configuration 1: Multi-Asset Mars Relay 30
12.3. Routing Policies Tested 31
12.4. Methodology 31
12.5. Results 31
12.6. Real-Stack Data-Plane Validation (ION PWG Testbed) 32
12.7. Goals (per RFC 2026 Section 4.2.1) 34
12.8. Success Criteria (Met) 34

12.9. Failure Criteria (Not Triggered) 35
12.10. Deferred Configurations 35
12.11. Open Experimental Questions 35
13. Reference Implementation 37
13.1. Repository 37
13.2. Reproducing the Experiment Results 37
13.3. Relationship to Updates of BPv7 38
14. Normative References 38
15. Informative References 39
Appendix A. Mars Mission Scenario 40
Appendix B. Change Log 40
B.1. draft-perry-dtn-cpb-00 (July 2026) 40
Author's Address 41

1. Introduction

Delay-tolerant networks (DTNs) often face uncertain contact schedules, such as during Mars mission disruptions, where standard BPv7 routing like Contact Graph Routing (CGR) may delay delivery due to reliance on fixed contact plans. Probabilistic routing metadata enables adaptive forwarding decisions by communicating contact likelihood estimates between nodes.

This specification defines a bundle extension block (the Contact Probability Block, or CPB) for carrying probabilistic contact metadata, enabling efficient routing decisions in DTNs for applications like space exploration communications.

1.1. Problem Statement (IPNSIG SSI Architecture, Section 4.2)

The Interplanetary Networking Special Interest Group (IPNSIG), the Interplanetary Chapter of the Internet Society, published "Solar System Internet Architecture and Governance — from the Moon to Mars and beyond" in September 2023 [IPNSIG2023]. Section 4.2 ("Routing and Forwarding") of that document frames the routing problem this specification addresses:

> The routing and forwarding procedures of the terrestrial Internet would not adapt well in the Solar System Internet, as topological state information could not be provided in a timely manner due to the lengthy signal propagation delays, frequent lapses of connectivity, and dynamic topology... New technology is needed.

The first "Considerations" bullet of IPNSIG Section 4.2 directly invites the design approach taken by this specification:

> By what delay-tolerant mechanism(s) does this technology obtain the information on which it decides which next-hop node to forward a given bundle to? Does it do so by route computation, by scoring and/or preferential ranking of neighboring nodes, or in some other way?

The Contact Probability Block defined here is precisely such a "scoring and/or preferential ranking" signal, carried in-bundle as metadata for forwarding nodes to consult. The CPB design responds directly to two of IPNSIG Section 4.2's four numbered Recommendations:

Recommendation 1 (Autonomy and Automation): "routing and forwarding technologies are needed that do not require continuous human intervention or management from Earth." CPB enables autonomous, in-bundle probability annotation that each forwarder consults locally, without round-trips to Earth.

Recommendation 2 (Standards): "Autonomy, in turn, requires standard methods of: propagating whatever information is needed to support route computation; utilizing that information in common ways that assure coherent forwarding." CPB standardizes both the wire format for propagating probability information (Section 3.4) and the precedence and matching rules for utilizing it (Section 3.6, Section 4.2).

Section 5 positions CPB relative to the routing-protocol families catalogued in IPNSIG Section 4.2 and its Appendix A: PRoPHET [RFC6693], Spray and Wait [Spyropoulos05], Schedule-Aware Bundle Routing (SABR) [SABR], Contact Graph Routing (CGR) [CGR], and Opportunistic CGR with Uncertain Contact Plans (CGR-UCoP) [Fraire18]. CPB is a cross-protocol metadata layer, not a replacement for any of these algorithms; each can consume or produce CPB data.

1.2. Motivation

Standard DTN routing protocols operate with deterministic contact schedules or historical encounter statistics. However, in highly disrupted networks:

- Contact schedules change unpredictably (dust storms on Mars, solar weather, orbital variations)
- Historical data becomes stale
- Coordinated path computation elements (PCEs) can estimate probabilities but need a way to communicate them to forwarding nodes
- Adaptive routing decisions require explicit probability metadata at the bundle level

By embedding probability estimates directly in bundles as a standard extension block, this specification enables all DTN routing protocols to benefit from more informed forwarding decisions.

1.3. Requirements Language

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

1.4. Terminology

Contact Probability The likelihood of successful bundle delivery to a destination node via a particular next hop or path, expressed as a value in [0.0, 1.0].

Contact Probability Block (CPB) A bundle extension block that carries contact probability metadata, routing hints, and temporal context.

Path Probability A probability estimate specific to a particular next-hop neighbor or outgoing interface.

Default Probability An overall or baseline probability estimate when per-path probabilities are unavailable.

Metric Type An identifier distinguishing the semantic meaning of the probability value (e.g., PRoPHET delivery predictability, CGR contact confidence, MaxProp path reliability).

Validity Duration The time period (in seconds) after creation during which the probability estimate remains usable for routing decisions.

1.5. Notation Conventions

This document uses three distinct notations for describing data structures, following the conventions of other DTN Working Group drafts:

CDDL (Concise Data Definition Language) Formal schema definitions per [RFC8610]. Listings are labeled "type: cddl". CDDL defines the structural constraints that implementations **MUST** conform to.

CBOR Extended Diagnostic Notation (EDN) Human-readable representations of CBOR data per [RFC8610] Appendix G. Listings are labeled "type: cbor-diag". EDN examples show concrete data values in readable form.

Annotated Hex Binary Base-16 encoded wire format with inline comments. Listings are labeled "type: hex". These show the exact byte sequences that appear on the wire.

2. Architectural Considerations

This section explains the architectural rationale for using bundle extension blocks rather than URI query parameters for probabilistic routing metadata.

2.1. Why NOT URI Query Parameters

An earlier design considered encoding probability as a URI query parameter in the destination EID:

```
<CODE BEGINS>
ipn:100.1?prob=0.75

<CODE ENDS>
```

That construction is architecturally flawed: it violates fundamental DTN principles for the following reasons:

2.1.1. Endpoint Identifier Immutability

Endpoint identifiers (EIDs) are immutable addresses that uniquely identify bundle destinations. If routing metadata were embedded in EIDs:

- Intermediate routers would need to modify destination addresses to update or strip probability parameters
- This violates the end-to-end principle and the Bundle Protocol specification [RFC9171]
- Routers cannot determine whether ipn:100.1?prob=0.75 and ipn:100.1?prob=0.5 represent the same or different endpoints
- Destination nodes would receive bundles with varying EID formats depending on router processing
- Security mechanisms binding bundles to specific EIDs (BPSec [RFC9172]) would break if routers modified destinations
- EID-based duplicate detection would fail when the same logical endpoint has multiple representations
- Custody transfer mechanisms would malfunction

2.1.2. Router Modification Problem

If probability were in the destination EID, the following scenario occurs:

1. Router A receives a bundle destined to ipn:100.1?prob=0.75
2. Router A consults the probability for forwarding decisions
3. Router A must now decide:
   - Forward as-is with ?prob=0.75 (but this value may be stale/incorrect for the next hop)
   - Strip the parameter to ipn:100.1 (modifying the destination address)
   - Update the parameter to a new value, e.g., ?prob=0.6 (also modifying the destination address)
4. Any modification violates the immutability principle and breaks downstream systems

This architectural violation is unacceptable in the DTN model.

2.2. Solution: Bundle Extension Blocks

This specification resolves the architectural conflict by carrying probabilistic metadata in bundle extension blocks, completely separate from endpoint identifiers:

- Destination EID remains immutable and unchanged: ipn:100.1
- Routing metadata travels in extension blocks (the Contact Probability Block)
- Routers can freely add, remove, or modify metadata without affecting the destination EID
- Security is maintained independently using Bundle Integrity Blocks (BIBs)
- Legacy nodes safely ignore unknown extension blocks without affecting bundle delivery

This design maintains clean architectural separation between:

- Addressing (EIDs): immutable, end-to-end
- Routing Hints (CPB metadata): mutable, hop-by-hop

3. Contact Probability Block

3.1. Overview

The Contact Probability Block (CPB) is a standard BPv7 bundle extension block that carries contact probability metadata. Its purpose is to communicate probability estimates from source nodes, path computation elements (PCEs), or intermediate routers to downstream forwarding nodes.

3.2. Block Structure

The CPB has the standard bundle extension block structure defined in [RFC9171]. The canonical block structure is a CBOR array per RFC 9171 Section 4.3.2:

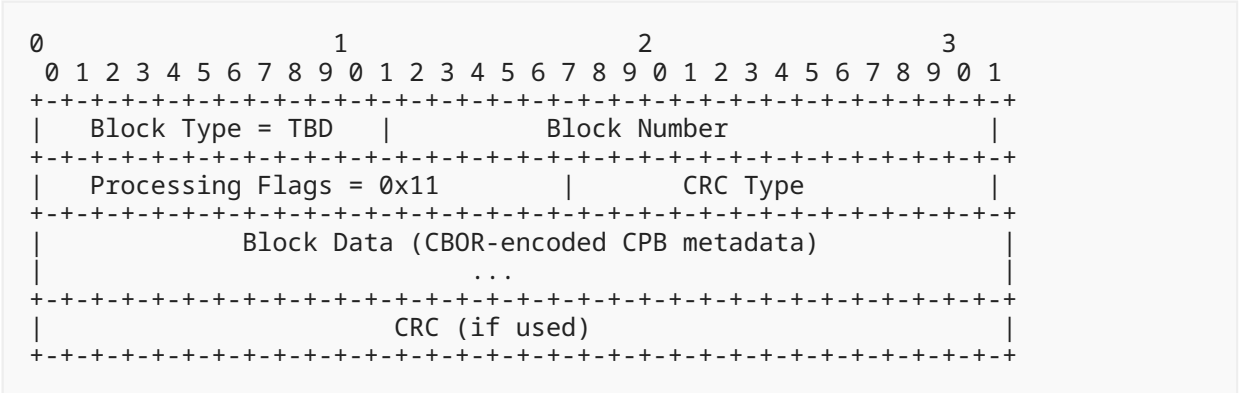

Figure 1: Bundle Extension Block Format (Octet Diagram)

The block-type-specific data (BTSD) field is a CBOR byte string (bstr) whose content is a CBOR-encoded cpb-data map. This byte-string embedding follows RFC 9171's canonical block structure, where the BTSD type is bstr (not any). The formal CDDL binding follows below.

The following annotated hex shows a concrete CPB with prob=0.75 for destination node 100:

```
<CODE BEGINS>
18 C8                ; type=200 (exp per RFC9171 &#167;9.1)
02                   ; num=2
11                   ; flags=0x11 (replicate; discard-if-unproc)
00                   ; crc-type=0 (no CRC; CRC field omitted)
57                   ; BTSD: bstr (23 bytes)
   A4                ; map with 4 entries
   00 F93A00         ;   0: prob=0.75 (float16)
   01 81             ;   1: array of 1 path entry
      82             ;     [node, prob]
      18 64          ;       node=100
      F93C00         ;       prob=1.0
   02 1A 00F73A80    ;   2: timestamp (CPB epoch seconds)
   04 19 0E10        ;   4: validity=3600s

<CODE ENDS>
```

Figure 2: CPB Concrete Example (type: hex)

The CBOR data structure in the block-data field is defined as follows. The BTSD is byte-string-embedded per RFC 9171's canonical block structure, using the CDDL socket and generic rules from RFC 9171 Section 4.3.2:

```
<CODE BEGINS>
; Integration with RFC 9171 canonical block structure.
; The BTSD is a bstr whose content is CBOR-encoded cpb-data.
; Hop-by-hop CPB MUST stay readable; do not BCB-encrypt (Sec 8.3.3).
; Plain bstr alt = RFC 9171 embedded-cbor (opaque BTSD).

$extension-block /=
    extension-block-use<200, (bstr .cbor cpb-data / bstr)>

<CODE ENDS>
```

Figure 3: CPB Extension Block Binding (type: cddl)

```
<CODE BEGINS>
cpb-data = {
    ? 0: probability,          ; Default/overall probability
    ? 1: [* path-entry],       ; per-path (SHOULD limit size; Sec 3.4)
    ? 2: uint,                 ; Timestamp (CPB epoch seconds)
    ? 3: bstr,                 ; Source PCE identifier (optional)
    ? 4: uint,                 ; Validity duration (seconds, TTL)
    ? 5: metric-type,          ; Metric type identifier
    ? 6: probability,          ; Confidence/variance estimate
    ? 7: uint,                 ; format version (1 = this spec)
    * (int .gt 7) => any       ; Extensibility (future fields)
}

probability = float            ; See Section 3.4 for encoding

; At least one of field 0 or field 1 SHOULD be present for a
; usable estimate. An empty map is valid CBOR but no routing signal.

path-entry = [
    next-hop: eid-reference,   ; Next-hop node identifier
    probability: float         ; Probability for this path
]

eid-reference = uint / tstr    ; node number or full EID text

metric-type = &(
    prophet-dp:      0,        ; PRoPHET delivery predictability
    cgr-confidence:  1,        ; CGR contact confidence
    maxprop-cost:    2,        ; MaxProp path cost (inverted)
    rapid-utility:   3,        ; RAPID utility metric
    generic:         4,        ; Unspecified/generic probability
) / uint                       ; Extensible via future assignment

<CODE ENDS>
```

Figure 4: CPB Data Structure (type: cddl)

Note on bstr .cbor embedding: The CDDL .cbor control operator ([RFC8610] Section 3.8.4) indicates that the byte string contains valid CBOR matching the cpb-data rule. The plain `bstr` alternative in the extension-block binding matches the RFC 9171 `embedded-cbor` pattern and accommodates BTSD that is opaque under a confidentiality context applied to the enclosing block structure. For hop-by-hop routing use, CPB MUST remain readable (see Section 8.3.3); do not BCB-encrypt the CPB in that deployment mode.

Field Semantics:

Field 0: Default Probability An overall probability estimate when per-path data is unavailable. Range [0.0, 1.0].

Field 1: Per-Path Array An array of [next-hop, probability] tuples. Senders **SHOULD NOT** include more than 8 entries. Receivers on bandwidth- or memory-constrained links **SHOULD** enforce a local policy limit (8 entries is **RECOMMENDED**) and **MAY** treat a CPB exceeding the local limit as malformed for that deployment, logging per [RFC9675] Section 4.3. The CDDL production uses a bound of 8 as an informative size

recommendation for typical constrained-link use cases; the normative limit is the SHOULD language above. Receivers and senders on unconstrained links MAY use larger arrays.

Field 2: Timestamp Creation time of this CPB estimate, in whole seconds since 2000-01-01 00:00:00 UTC (the same epoch origin as DTN Time in [RFC9171], but this field is defined in **seconds**, not the millisecond DTN Time unit of RFC 9171 Section 4.2.6). Receivers **MUST** interpret field 2 and field 4 in this same second-based unit. Enables receivers to compute age and apply local decay policies.

Field 3: Source PCE Identifier An opaque byte string identifying the entity (e.g., a PCE or router) that computed the probability values in this CPB. This field supports trust and policy decisions (see Section 8). No particular encoding or registry is defined. The value is meaningful only within the administrative scope(s) that trust the producer. Deployments that do not require source attribution **MAY** omit this field.

Field 4: Validity Duration (TTL) The number of seconds after the timestamp during which this probability estimate remains usable. When the sum of timestamp and validity duration is less than the current time, the CPB is expired and **SHOULD** be ignored in full for routing (all probability fields, including per-path entries). If omitted, implementations **SHOULD** apply deployment-specific age thresholds (Section 4.1).

Field 5: Metric Type Identifies the semantic meaning of the probability value. When absent, implementations **SHOULD** treat the value as generic (type 4).

Field 6: Confidence An optional float value in [0.0, 1.0] representing the confidence or certainty in the probability estimate. A confidence of 0.9 with P = 0.5 (based on 1000 observations) carries more weight than confidence of 0.1 with P = 0.5 (based on 2 observations). Routing algorithms **MAY** use this for risk-aware forwarding.

Field 7: Version CPB format version number. If absent or set to 1, the structure and semantics are as defined in this specification. Receivers **SHOULD** process all recognized fields when encountering a higher version number and **MAY** ignore unrecognized higher fields or additional semantics defined in future specifications. Senders using only the fields defined here **SHOULD** omit the version field or set it to 1.

Receivers encountering an unrecognized metric-type value **SHOULD** ignore that CPB for routing unless local policy explicitly maps the codepoint. They **MUST NOT** silently coerce an unknown metric-type into metric-type 4 (generic) for automated arithmetic or ranking. Senders **MUST** use only registered values defined in the IANA "CPB Metric Types" registry (see Section 9) unless operating in a private or experimental context.

3.2.1. Block Processing Flags

The CPB **MUST** set the block processing control flags ([RFC9171] Section 4.4.1) as follows:

- **Bit 0 (0x01): Replicate in all fragments:** Implementers **MUST** set this bit so that if a bundle with a CPB is fragmented during transmission, each fragment carries a copy of the CPB.
- **Bit 4 (0x10): Discard if can't process:** Implementers **MUST** set this bit because the CPB is advisory routing metadata, not critical to bundle delivery.
- **Bits 1-3, 5-7:** Leave as 0 (reserved).

In practice, both flags set means the block processing flags byte is set to 0x11 (binary 00010001).

Path-entry handling under fragmentation: When replicating a CPB into a fragment, the fragmenting node **MAY** prune path-entry elements (field 1) that are known to be unreachable in the fragment's remaining path, but **MUST NOT** alter the probability values of retained entries. The default-probability field (0) and metric-type field (5) **MUST NOT** be modified during fragmentation.

3.2.2. Cyclic Redundancy Check (CRC) Handling

The block-data CRC field ([RFC9171] Section 4.4.1) is optional:

- **No CRC (recommended):** Set crc-type to 0 and omit the CRC field from the canonical block array (five elements total), per [RFC9171] Section 4.3.2. This is the default and minimizes overhead. Implementations **MUST NOT** emit a CBOR null CRC element when crc-type is 0.
- **Optional CRC:** Implementations **MAY** include CRC type 1 (X.25 CRC-16) or type 2 (CRC32C / Castagnoli) as defined by [RFC9171].

Implementations **SHOULD** prefer BPSec BIBs (Section 8.3) over CRCs for integrity, as BIBs provide both authentication and integrity.

3.2.3. Hop Count Block Co-Presence

Bundles carrying a CPB **SHOULD** also carry a Hop Count block (block type 10, [RFC9171] Section 4.4.2). The Hop Count block provides a loop termination guarantee independent of probability-based routing decisions.

3.3. Block Type Code

The bundle extension block format ([RFC9171]) requires each extension block to have a type code that identifies what kind of block it is.

3.3.1. IANA Registration

This specification requests IANA to assign a permanent block type code from the "Bundle Block Types" registry (created by [RFC9171]). Until IANA assignment:

- Implementations **MAY** use values in the 192-254 range for experimental/private use
- The example hex dump uses code 200 (0xC8) for illustration
- Production deployments **SHOULD** use the IANA-assigned code

3.3.2. Backwards Compatibility

Routers that don't recognize the CPB block type code will:

1. Check the block processing control flags (byte 2 in the extension block)
2. If flags indicate "discard if can't process", drop the block silently
3. If flags indicate "remove if can't process", strip it before forwarding
4. Continue routing the bundle using remaining blocks (backwards compatible)

3.4. CBOR Encoding of Probability Values

The CDDL schema (Section 3.2) defines probability fields as float rather than constraining to a specific precision, because enforcing float16 in CDDL is difficult for encoders to implement in practice. Instead, the precision requirement is expressed as a prose recommendation:

Implementations **SHOULD** choose probability values that are exactly representable in IEEE 754 binary16 (half-precision). All values in [0.0, 1.0] at routing-grade precision (~0.001 granularity from the 10-bit mantissa) are representable in binary16. When a value is representable in float16, [RFC8949] Section 4.2.1's deterministic encoding requirement ensures it will be encoded in 3 bytes.

- **Range:** [0.0, 1.0], sufficient for all probability values
- **Precision:** Approximately 3 decimal digits, adequate for routing decisions
- **Overhead:** 3 bytes per value when float16 is used
- **Standard:** Defined in [RFC8949] (CBOR specification)

Implementations that produce probability values not representable in float16 will incur 5 bytes (float32) or 9 bytes (float64) per value under deterministic encoding. Receivers **MUST** accept any valid CBOR float encoding and clamp the decoded value to [0.0, 1.0].

Bandwidth Considerations and Array Limits: A CPB with per-path probabilities to multiple relays adds overhead (~4-5 bytes per entry). To mitigate DoS risks from oversized arrays on low-bandwidth or constrained links, senders **SHOULD NOT** produce per-path arrays larger than 8 entries. Receivers in constrained deployments **SHOULD** enforce a local limit (default 8) and treat excess entries as malformed for that link, logging the event per [RFC9675] Section 4.3. This guidance is normative for resource protection; the CDDL schema provides a default bound of 8.

3.4.1. Invalid Float Handling

Implementations **MUST** handle invalid or out-of-range probability values as follows:

- **NaN (0xF97E00): MUST** be rejected as malformed.
- **+/-Infinity (0xF97C00, 0xF9FC00): MUST** be rejected as malformed.
- **Negative values:** Treat as 0.0 (invalid probability).
- **Values > 1.0:** Treat as 1.0 (certainty).
- **Subnormal values: SHOULD** be flushed to zero for embedded FPU compatibility.
- **Malformed CBOR:** Discard the CPB block and continue processing the bundle.

In all error cases, implementations **SHOULD** log the error (including source, timestamp, and offending value) for debugging. Malformed CBOR **MUST** never cause bundle deletion or administrative action; the CPB is advisory and bundle processing continues regardless.

3.4.2. Encoding Details

When a probability value is representable in half-precision, the CBOR encoding uses major type 7 with additional information value 25 (16-bit IEEE 754 binary16):

```
Major Type:      7 (floating-point or simple)
Additional Info: 25 (16-bit IEEE 754 float)
Total Bytes:     3 (tag byte 0xF9 + 2 float bytes)
```

Figure 5: CBOR Encoding: Major Type 7, Additional Info 25

3.4.3. Hex Encoding Table

Probability	Binary16	CBOR Hex	Bytes
0.0	0x0000	0xF90000	3
0.25	0x3400	0xF93400	3
0.5	0x3800	0xF93800	3
0.75	0x3A00	0xF93A00	3
0.95	0x3B9A	0xF93B9A	3
1.0	0x3C00	0xF93C00	3

Table 1: CBOR Encodings for Common Probability Values

3.5. Metric-Type Semantics and Cross-Metric Comparison

The probability values carried in a CPB are dimensionless numbers in [0.0, 1.0], but their *semantics* differ substantially across routing protocols. The metric-type field (field 5, Section 3.2) distinguishes these semantics. This subsection specifies the constraints on combining values across metric types.

The defined metric types carry the following distinct semantics:

metric-type 0 (prophet-dp) PRoPHET delivery predictability P_(A,B) as defined in [RFC6693] Section 2.1.2: an empirical encounter-history statistic in [0.0, 1.0]. Reflects observed past co-location behavior (with aging and optional transitivity applied *before* encoding), not a scheduled-contact success probability. Values in a CPB **MUST** already be aged at the producing node (Equation 2 of RFC 6693); receivers **MUST NOT** re-apply a second aging constant unless they recompute predictabilities from their own encounter base. Default probability (field 0) **SHOULD** be interpreted as the producer's P_(producer, dest) toward the bundle destination. When field 1 is present, each path entry for next-hop NH **SHOULD** carry the producer's estimate of P_(NH, dest) (the delivery predictability of that next hop toward the destination), not a repeat of P_(producer, dest) (see Section 5.1).

metric-type 1 (cgr-confidence) Per-contact or per-next-hop success probability against a known contact schedule (or contact-plan edge). Reflects forward-looking confidence that a planned contact occurs as scheduled (link availability, weather, pointing). Contact start/end times, one-way light time, and residual volume remain in the contact plan, not in CPB; CPB carries only the scalar confidence the plan or local estimator associates with a hop or default path.

metric-type 2 (maxprop-cost) Delivery-likelihood proxy derived from MaxProp-style path cost. Producers **MUST** map lower MaxProp cost to higher CPB value in [0.0, 1.0] using a documented local inversion (for example, 1/(1+cost) or a rank-normalized score). The inversion is lossy; original cost magnitude and multi-hop cost vectors are not recoverable from the CPB value. Hop-lists used for loop avoidance and delete-ACKs remain local MaxProp state, not CPB fields.

metric-type 3 (rapid-utility) Normalized per-packet utility from a RAPID-style optimizer, scaled by the producer into [0.0, 1.0]. This is an optimization-derived score (often over delay or resource constraints), not a calibrated delivery probability. Unnormalized utilities, delay distributions, and resource inventories are out of band; if a deployment cannot normalize utility into [0.0, 1.0] honestly, it **SHOULD** register a new metric-type or use metric-type 4 only as advisory.

metric-type 4 (generic) Unspecified semantic. The producer makes no claim beyond "value in [0.0, 1.0]". Consumers **SHOULD NOT** assume any specific interpretation. Suitable for experimental social or hybrid scores (centrality, similarity) until a dedicated metric-type is registered.

Cross-metric arithmetic is prohibited. A router **MUST NOT** combine values from different metric types via arithmetic (addition, multiplication, weighted averaging, etc.) and treat the result as a meaningful probability. A PRoPHET predictability of 0.7 and a CGR confidence of 0.7 are not interchangeable inputs to the same routing decision. Specifically:

- A router implementing a confidence-weighted policy that may also use rates that consumes CPB data (for example, a policy whose cost function resembles formulations in prior work such as CGR-UCoP [Fraire18]) **MUST** use only metric-type 1 (cgr-confidence) values, and **MUST** treat path-entries of other metric types as if absent.
- A router applying PRoPHET-style transitivity rules **MUST** use only metric-type 0 (prophet-dp) values, and similarly ignore others.
- A router consuming metric-type 4 (generic) **SHOULD** treat the value as advisory only and **MUST NOT** use it as the sole input to a routing decision.

Mixed-metric CPBs: A single CPB instance carries one metric-type (field 5). A bundle **MAY** carry multiple CPBs with different metric types simultaneously (for example, a CPB with cgr-confidence from a PCE and a separate CPB with prophet-dp accumulated at intermediate hops). The precedence rules of Section 3.6 apply within a metric type; across metric types, each CPB is consumed only by routers implementing that metric's algorithm. A router that does not understand a particular metric type **MUST** ignore that CPB and continue processing.

Metric-type registry policy: Future routing protocols defining new probability semantics **SHOULD** register a new metric-type code (see Section 9) rather than overloading an existing one. The CPB Metric Types sub-registry exists precisely to keep semantically distinct values separable on the wire.

Calibration and normalization: This document does not specify normalization between metric types because cross-metric equivalence has no rigorous theoretical foundation in the published DTN literature. Implementations that wish to combine evidence from multiple metrics for ranking purposes **SHOULD** do so using a documented, deployment-specific policy and **SHOULD NOT** rely on numerical comparability across types. This area is identified as an open research question in Section 12.11.

3.6. Multiple CPBs in One Bundle

A bundle may accumulate multiple Contact Probability Blocks as it traverses the network. When a bundle contains multiple CPBs with conflicting probabilities for the same destination, implementations **SHOULD** use the following precedence:

1. Most recent timestamp (highest value)
2. If timestamps equal, trust the source (implementation-defined trust policy)
3. If timestamps are equal AND sources are equally trusted, prefer the CPB with the more specific metric-type (e.g., cgr-confidence over generic) for the current routing protocol
4. Default behavior: use the last encountered CPB (highest bundle block number)

Implementations **MUST NOT** fail or misroute when encountering multiple CPBs.

```
<CODE BEGINS>
/ CBOR Extended Diagnostic Notation (EDN) /
{
    0: 0.75,                    / Default probability /
    1: [
        [300, 0.5],             / To relay node 300: 0.5 /
        [100, 1.0]              / To orbiter node 100: 1.0 /
    ],
    5: 1                        / metric-type: cgr-confidence /
}

<CODE ENDS>
```

Figure 6: CPB with Per-Path Probabilities (type: cbor-diag)

```
<CODE BEGINS>
A3                  ; map with 3 entries
   00 F93A00        ; key 0: default prob = 0.75
   01 82            ; key 1: array with 2 path entries
      82            ; path entry [node, prob]
      19 012C F93800 ;   node 300, prob 0.5
      18 64 F93C00   ;   node 100, prob 1.0
   05 01            ; key 5: metric-type = cgr-confidence

<CODE ENDS>
```

Figure 7: CPB Per-Path Wire Encoding (type: hex)

3.6.1. Per-Path Matching Rules

When a router consults the per-path array:

1. Match the next-hop node/EID against path-entry keys
2. If exact match found, use that probability
3. If no match found, use default probability (field 0)

4. If no default probability is defined, treat the CPB as providing no default; fall back to local routing state (do not invent a universal 0.5)

EID-references in the path-entry array **MUST** match the destination bundle's URI scheme. For IPN bundles, node numbers are encoded as CBOR unsigned integers (per [RFC9758]'s updated ipn URI scheme); for other schemes, encode the full EID as CBOR text string.

Scheme Mismatch Handling: When the next-hop scheme does not match the CPB's path-entry scheme, implementations fall back to the default probability (field 0); if no default is defined, treat the estimate as absent and use local routing state. Implementations **SHOULD** log the scheme mismatch for OAM correlation per [RFC9675] Section 4.3.

4. Operational Semantics

4.1. Creation

The Contact Probability Block is created by:

- The source application (using historical or predicted contact data)
- A path computation element (PCE) (using network-wide routing state)
- The first-hop router (using local contact statistics)

Implementations **SHOULD** include a timestamp (field 2) and a validity duration (field 4). Timestamps **MUST** be expressed in seconds since 2000-01-01 00:00:00 UTC (DTN epoch, per [RFC9171] Section 4.3.1).

In networks with significant propagation delays, clock synchronization is challenging. When validating a CPB timestamp, implementations **SHOULD** apply tolerance for clock drift and light-time delays:

- **Forward skew tolerance:** Apply a configurable drift threshold (default: +/-600 seconds). CPBs with timestamps > current time + drift threshold **MUST** be discarded.
- **Backward skew/age threshold:** CPBs with timestamps < current time - max_age **MUST** be discarded (default max_age: 86400s interplanetary, 3600s terrestrial).
- **Deployment-specific adjustment:** Networks **SHOULD** define explicit max_drift and max_age per link characteristics.

4.2. Processing by Intermediate Routers

When a router processes a bundle containing a CPB:

1. **Read:** Consult the CPB's default or per-path probability values.
2. **Decide:** Use the probability as an input to routing decisions.
3. **Update (optional):** If the router has fresher probability data, it **SHOULD** append a new CPB block to the bundle (preserves any end-to-end integrity signatures protecting the original block).
4. **Forward:** Transmit the bundle with the CPB(s) to the next hop.

4.3. Usage in Routing Algorithms

Routing algorithms **MAY** use the CPB in several ways:

- **Local state preference:** Nodes that maintain native protocol state (PRoPHET RIB, CGR contact plan, MaxProp tables) **SHOULD** prefer that local state over foreign CPB values unless local policy explicitly imports CPB as authoritative for a domain or metric-type.
- **Threshold-based:** Forward if prob > threshold
- **Weighted selection:** Choose proportionally to probability
- **Replication target ranking:** When a local spray/epidemic policy already owns a remaining copy count L, CPB probabilities **MAY** rank candidate relays. CPB does not encode L and **MUST NOT** be read as defining fanout as ceil(1/prob) (see Section 5.4).
- **Priority:** Prioritize higher-probability bundles during congestion
- **Risk-aware:** Weight decisions by both probability and confidence (field 6)

4.4. Stability Considerations and Feedback Dynamics

Probability-aware forwarding creates feedback between routing decisions and observed network behavior. Naive use of CPB data can produce instability that confidence-blind routing avoids. Three dynamics are particularly relevant:

Herding When multiple sources receive identical high-confidence CPB advertisements for the same relay, they all forward to that relay simultaneously. The relay congests, its actual delivery probability drops, but in-flight CPBs continue advertising the stale high confidence. Traffic remains herded until updated CPBs propagate, which in DTN timeframes can take hours.

Route oscillation If a router switches preferences whenever CPB values cross a threshold, small probability fluctuations near the threshold cause traffic to oscillate between routes. Each switch may itself generate new CPB observations that further perturb the values.

Synchronized congestion collapse In dense topologies, simultaneous reaction by many routers to the same CPB update can produce correlated congestion events. This is structurally similar to TCP synchronization in IP networks but with much longer recovery cycles due to DTN propagation delays.

Implementations **SHOULD** mitigate these dynamics. The following techniques are **RECOMMENDED**:

- **Hysteresis:** Use distinct enter and exit thresholds (e.g., switch *to* a route at $p > 0.75$, switch *away* only at $p < 0.60$). Avoids threshold-crossing chatter on small fluctuations.
- **Rate limiting:** Cap the frequency of routing preference changes per (source, destination) pair (e.g., at most one switch per contact opportunity, not per bundle).
- **Randomized tie-breaking:** When multiple routes have CPB probabilities within a configurable epsilon (default: 0.05), select randomly rather than deterministically by lowest EID. Breaks herding without sacrificing the high-probability choice.
- **Probability decay under load:** Maintain a local estimate of congestion on each next-hop and apply a load-aware discount to the received CPB probability before using it in routing decisions. Closes the feedback loop locally rather than waiting for upstream CPB refresh.

- **Bounded credulity:** Cap how heavily any single CPB can influence the local routing decision relative to the router's own observations. Specific weights are deployment-dependent.

A formal control-theoretic stability analysis of CPB-driven feedback dynamics is not provided in this document; this is identified as an open research question in Section 12.11. Implementations that omit all of the above mitigations **SHOULD NOT** be deployed in production DTNs where congestion or herding could degrade service.

5. Relationship to Existing DTN Routing Protocols

This section positions CPB relative to the existing DTN routing-protocol families catalogued in IPNSIG Section 4.2 and its Appendix A ("Routing and Forwarding State of the Art").

5.1. PRoPHET (RFC 6693)

PRoPHET [RFC6693] is an epidemic protocol with strict pruning. Each node maintains delivery predictabilities P_(A,B) updated on encounters (Eq. 1), aged over time (Eq. 2), and optionally improved by transitivity via an intermediate (Eq. 3). During a contact, peers exchange a Routing Information Base (RIB) of destination-scoped predictabilities, then apply a local forwarding strategy (for example GRTR / GTMX) and queueing policy. Positive delivery acknowledgments prune residual copies.

What CPB carries for PRoPHET: metric-type 0 values that a producer has already computed (aged, optionally transitive) for the bundle destination and/or candidate next hops. CPB is an optional in-bundle hint so downstream nodes that do not speak the full PRoPHET RIB exchange can still see a snapshot of the producer's P.

What stays local (not CPB): aging constants (gamma, K), P_encounter / beta parameters, the full RIB dictionary, Hello / anti-entropy session state, bundle offer and response TLVs, custody policy, and named forwarding/queueing strategies. CPB does not replace the PRoPHET control plane in RFC 6693 Sections 3–5.

5.2. MaxProp

MaxProp estimates delivery likelihood from encounter history, prefers low path-cost relays, prioritizes new packets by hop count, uses hop-lists to avoid cycles, and floods delete-ACKs so peers can drop delivered bundles. Buffer drop order is tightly coupled to the same likelihood metric used for forwarding.

What CPB carries: metric-type 2 as a lossy [0.0, 1.0] inversion of path cost / delivery likelihood so foreign nodes can rank next hops without MaxProp's full peer exchange.

What stays local: cost vectors, hop-lists, delete-ACK flooding, and the dual use of likelihood for queue-drop priority (though a CPB-aware node **MAY** use metric-type 2 values as one input to local drop ranking; see Section 5.9).

5.3. RAPID

RAPID treats routing as a resource allocation problem: it computes a per-packet utility (often over expected delay under bandwidth and storage constraints) and replicates in decreasing marginal-utility order. Peers may exchange delay samples or related metadata to refine utilities.

What CPB carries: metric-type 3 only after the producer normalizes utility into [0.0, 1.0]. CPB does not carry raw delay distributions, residual bandwidth, or multi-objective utility vectors.

5.4. Spray-and-Wait Family

Spray-and-Wait [Spyropoulos05] limits replication to L copies. Binary spraying and Spray-and-Focus variants choose relays uniformly or by a utility in the focus phase. The remaining copy count is the primary control state.

What CPB carries: optional ranking hints (metric-type 0/1/2/3/4) for *whom* to spray to when a node chooses non-uniform targets.

What stays local / out of scope for CPB: the integer L remaining-copy counter and spray-phase state. Those are replication-control metadata, not probabilities; they need a different extension or local header if standardized later. CPB does not encode L.

5.5. Epidemic and Summary-Vector Protocols

Classic epidemic routing and many DTN anti-entropy designs exchange summary vectors of buffered bundle IDs so peers can request missing copies. That inventory exchange is orthogonal to probability metadata. CPB neither replaces summary vectors nor carries bundle-set digests. A node may still attach a CPB when forwarding a copy so receivers learn the forwarder's confidence snapshot.

5.6. Contact Graph Routing (CGR)

Contact Graph Routing and SABR-style schedule-aware routing compute earliest-arrival (or related) routes from a contact plan: contact intervals, data rates, one-way light times, and residual volume. CGR-UCoP [Fraire18] and related work add contact-failure probabilities to that plan so cost can trade latency against reliability.

What CPB carries: metric-type 1 confidences for default or per-next-hop edges so in-bundle state can move with the traffic when contact-plan dissemination is slow or partial.

What stays in the contact plan: time windows, rates, OWLT, and residual capacity. CPB is not a substitute contact plan. The closest architectural precedent for in-band routing metadata is the CGR Extension Block [CGREB]. Section 12 compares confidence-blind earliest arrival to one confidence-weighted consumer of ground-truth confidences of the CPB class under stochastic contact failures; CGR-UCoP is prior art for simulation, not the definition of the block.

5.7. Social and Hybrid Metrics

Protocols such as BubbleRap, SimBet, and related social DTN routers use centrality, community labels, or similarity rather than pure encounter probability. Those scores may be published as metric-type 4 (generic) with local documentation, or as new metric-type codepoints via the IANA sub-registry in Section 9. CPB still forbids mixing them arithmetically with prophet-dp or cgr-confidence.

5.8. DTNMA (RFC 9675)

[RFC9675] defines a DTN management architecture orthogonal to routing. A DTNMA agent could monitor CPB-related metrics (frequency of stale CPBs, scheme mismatches, BIB verification failures) for network health assessment.

5.9. Summary: CPB as Cross-Cutting Metadata

Across the families above, a recurring split appears:

- **In-bundle scalar quality (CPB):** a [0.0, 1.0] score with an explicit metric-type, optional per-next-hop list, timestamp, and validity.
- **Control-plane state (not CPB):** encounter RIBs, contact plans, summary vectors, copy counts, hop-lists, delete-ACKs, Hello sessions, and strategy/queue policy parameters.

The Contact Probability Block is designed as that cross-protocol scalar layer rather than a replacement for any algorithm. Algorithms **MAY** consume CPB for forwarding rank, replication target choice, or buffer-drop priority, and **MAY** produce CPB for downstream nodes, without requiring the peer to implement the producer's full protocol.

6. Backwards Compatibility

- Nodes that do not recognize the CPB block type **MUST** ignore it and continue processing the bundle normally
- The bundle's destination, source, and other standard fields are unaffected by the CPB
- Legacy routing protocols (CGR, PRoPHET, MaxProp, RAPID, Spray-and-Wait) continue to function without the CPB
- The CPB is optional; bundles without it use standard routing behavior

7. Operational Considerations

This section collects deployment, sizing, and operational guidance for CPB. It draws from the processing rules in Section 4, the overhead analysis in Section 10, and the backwards-compatibility rules in Section 6.

7.1. When and Where to Emit CPBs

A CPB may be created by the source application (using historical or predicted contact data), a path computation element (PCE) (using network-wide state), or the first-hop router (using local contact statistics). Deployments **SHOULD** document their policy for which entity is authoritative for a given bundle class.

In environments with significant propagation delay (e.g., interplanetary), a PCE close to the source is often preferable so that the initial CPB reflects the most current available model. In low-delay terrestrial meshes, the first-hop router may have fresher local observations.

7.2. Bundle Store and Memory Sizing

If the strip-and-replace optimization in Section 3.6 is not used, bundles can accumulate multiple CPBs across hops. Each CPB occupies approximately 8-32 bytes of CBOR plus any BPSec BIB overhead. Implementations on memory-constrained nodes **SHOULD** bound the number of retained CPBs per bundle and apply the optional removal of earlier instances when appending a fresher CPB.

Bundle stores **SHOULD** account for the additional per-bundle overhead when dimensioning for CPB-enabled traffic.

7.3. Computational and Storage Overhead on Constrained Nodes

Decoding a CPB map (up to 8 path entries) requires CBOR parsing of roughly 32-100 bytes. On commodity hardware this is sub-millisecond; on microcontrollers using software floating-point it may be several milliseconds per CPB.

If the CPB is protected by a BPSec BIB, each consumption requires a signature or MAC verification. Implementations **SHOULD** cache verification results for the duration of bundle processing.

Route recomputation triggered by new CPB observations can be bounded by the hysteresis and rate-limiting techniques described in Section 4.1.

Implementations **SHOULD** publish concrete profiling numbers (CPU cycles per CPB, memory delta, etc.) alongside interoperability reports, per the guidance in [RFC7942].

7.4. Logging, Monitoring, and OAM

Nodes **SHOULD** log CPB-related events (reception of invalid floats, scheme mismatches, version numbers higher than supported, BIB verification failures on CPBs, etc.) using the mechanisms defined in [RFC9675] Section 4.3.

When a CPB is discarded due to age, future-dated timestamp, or malformed content, a management event **SHOULD** be generated to aid debugging of clock skew or misbehaving producers.

7.5. Partial Deployment

CPB is designed for incremental deployment. Nodes that do not recognize the block type pass it through unchanged (see Section 6). However, the routing quality improvement is only realized by CPB-aware forwarders. Deployments **SHOULD** monitor the fraction of CPB-aware nodes and may choose to emit CPBs only on segments where a useful number of consumers exist.

7.6. Congestion and Storage Pressure

Probability-aware routing tends to concentrate traffic on high-confidence routes. Under sustained load this can lead to congestion on preferred paths and storage pressure at intermediate nodes.

The mitigations in Section 4.1 (hysteresis on route changes, minimum interval between CPB-triggered recomputations, and explicit rate limiting) are **RECOMMENDED**. Operators **SHOULD** monitor queue depths and delivery ratios on high-confidence routes and be prepared to fall back to confidence-blind policies or to reduce CPB production rate during overload.

7.7. Clock Considerations

CPB timestamps are in DTN epoch seconds. In networks with light-time delay or unsynchronized clocks, the age and future-dated checks in Section 4.1 must use deployment-specific drift and max_age values. Interplanetary links commonly use larger tolerances (e.g., +/- 600 s drift, 86400 s max age) than terrestrial ones.

8. Security Considerations

8.1. Threat Model

- **Untrusted networks:** An attacker could inject false probability values to cause mis-routing (classic blackhole/sinkhole attraction via *inflated* metrics that draw traffic then drop it; selfish avoidance via deflated metrics)
- **Stale data:** A captured bundle with outdated probabilities could cause poor routing decisions
- **Inference attacks:** Observing probability values over time could reveal traffic patterns or network topology
- **Route poisoning by semi-trusted relays:** A relay that holds valid credentials within an administrative domain may still produce CPB advertisements that systematically bias routing decisions toward itself, away from a competitor, or against specific destinations
- **Replay of stale CPBs:** An attacker captures a recent valid (BIB-signed) CPB and replays it later when the underlying conditions have changed
- **Inter-domain trust escalation:** A CPB created by an entity trusted in one administrative domain crosses into a second domain where the producer is unknown; the second domain has no basis for trust evaluation

8.2. Trust Model

This document does not mandate a single trust model because DTN deployments span a wide range of administrative scenarios — from a single-operator interplanetary relay network to multi-party terrestrial mesh deployments. Instead, this subsection identifies the trust questions every CPB deployment **MUST** answer in its local trust policy, and provides default guidance.

8.2.1. Who is Authorized to Append CPBs

By default, any node along the bundle's path **MAY** append a CPB. This is permissive and suitable for low-stakes deployments where the cost of bad routing is recoverable. For higher-stakes deployments, the local trust policy **SHOULD** restrict CPB authorship to a named set of authorized PCEs (identified by the source PCE identifier field, field 3), or to nodes possessing a specific BPSec credential. CPBs from unauthorized sources **SHOULD** be discarded, with the event logged per [RFC9675] Section 4.3.

8.2.2. Cryptographic Adjudication of Conflicting CPBs

When a bundle carries multiple CPBs from different sources with conflicting probabilities, the precedence rules of Section 3.6 select one. Those rules **SHOULD** be evaluated only after BIB verification of each CPB; unsigned or BIB-invalid CPBs **MUST** be excluded from the precedence calculation in trust-policy contexts that require authentication. A receiver **MUST NOT** apply the metric-type-specificity tie-breaker (Section 3.6 step 3) to elevate a less-trusted source over a more-trusted one; trust ranking takes precedence over metric specificity.

8.2.3. Downstream Handling of Upstream BIBs

A downstream router that cannot verify an upstream BIB (missing key material, expired credentials, unrecognized security context) has two acceptable behaviors:

- **Strict:** Treat the unverifiable CPB as absent for routing decisions, and forward the bundle unchanged so that downstream routers with the correct credentials may still consume it. This is **RECOMMENDED** as the default.
- **Permissive:** Use the unverified CPB advisorily, treating it as metric-type 4 (generic) regardless of its declared metric-type. This is **NOT RECOMMENDED** in adversarial environments but may be appropriate in cooperative single-operator deployments where credential rollout lags CPB adoption.

A router **MUST NOT** silently strip an unverifiable BIB-protected CPB and re-sign it with its own credentials. Doing so launders provenance and breaks the append-only trust chain. If a router needs to express a different probability estimate, it **MUST** append a new CPB with its own BIB rather than modifying or replacing the original.

8.2.4. Route Poisoning by Semi-Trusted Relays

A relay that is trusted enough to participate in the network, but not trusted to make routing decisions for the entire domain, can still degrade routing by producing biased CPB advertisements. Defenses against this attack model are imperfect at the CPB layer. Mitigations **SHOULD** include:

- Plausibility checks: a receiver **SHOULD** compare the asserted CPB probability against locally observed delivery statistics for the same next-hop. CPBs that deviate beyond a deployment-specific tolerance (default: factor of 2) should be discounted or discarded.
- Source reputation: trust policies **MAY** maintain per-source reputation scores derived from historical accuracy, and weight CPBs by source reputation when consuming them. CPB itself does not specify a reputation mechanism; the DTN Management Architecture ([RFC9675]) is the appropriate vehicle.
- Bounded influence (Section 4.4): cap how much any single CPB can shift the local routing decision relative to the router's own observations.

None of these mitigations defeat a sufficiently capable adversary. They raise the cost of route poisoning and make casual misbehavior more detectable. Trust frameworks that fully close this gap (e.g., formal reputation aggregation or attestation chains) are out of scope for this document.

8.2.5. Inter-Domain Trust

When a bundle carrying a CPB crosses an administrative boundary, the receiving domain has no a priori basis for trusting the CPB's producer. Inter-domain trust **SHOULD** be established via one of the following approaches:

- A bilateral or multilateral trust agreement that designates which PCE identifiers and BIB credentials from the foreign domain are accepted
- A gateway router at the domain boundary that consumes the foreign CPB advisorily, validates it against local observations, and (optionally) appends a domain-native CPB expressing the gateway's translated assessment
- A policy of discarding all CPBs at the domain boundary, with the receiving domain regenerating CPBs internally from its own observations

This document does not standardize inter-domain trust establishment. Deployments **SHOULD** document their inter-domain CPB policy explicitly. This area is identified as an open question in Section 12.11.

8.2.6. Replay Resistance

The timestamp (field 2) and validity duration (field 4), combined with the skew tolerances defined in Section 4.1, provide replay resistance that scales with the configured age threshold. For stronger protection, BIB coverage must include the timestamp. An attacker who can rewrite the timestamp without invalidating the BIB defeats the mechanism. Implementations **MUST** ensure that the BIB security target list covers the entire CPB block.

8.3. Integrity Protection

For deployments in untrusted networks or where source authentication is required, the CPB **MUST** be protected using Bundle Integrity Blocks (BIBs) as defined in [RFC9172] (BPSec).

8.3.1. Source CPB with Append-Only Intermediates

The source creates a CPB and attaches a BIB protecting only that CPB's block number. Intermediate routers with fresher probability estimates append new CPB blocks without modifying the source's CPB.

8.3.2. Strip-and-Replace Model

As an alternative, implementations **MAY** use a strip-and-replace model where each forwarder validates the previous CPB, removes it, inserts a fresh CPB with its own probability estimate, and signs it with its own BIB. Deployments **SHOULD** choose one model per administrative domain and document the policy.

8.3.3. Prohibition on BCB Encryption

Implementations **MUST NOT** encrypt the CPB using a Block Confidentiality Block (BCB). Encrypting a routing-critical extension block renders it opaque to the forwarding nodes that need it. Integrity protection (BIB) is appropriate; confidentiality protection (BCB) is not.

8.4. Mitigations

- Signature verification before using CPB for critical routing decisions
- Timestamp validation (discard old or future-dated CPBs)

- Secondary validation against local contact statistics
- Conservative thresholds for critical payloads

8.5. Privacy Considerations

CPB metadata may reveal information about network topology and operational patterns. Per-path probability entries expose which next-hop nodes are reachable; the source PCE identifier (field 3) identifies the entity that computed the probability. Deployments in adversarial environments **SHOULD** consider omitting the source PCE identifier when source attribution is not required.

9. IANA Considerations

This document requests IANA to assign a new codepoint from the "Bundle Block Types" registry as defined in [RFC9171], Section 9.1, under the Specification Required registration policy ([RFC8126]).

Suggested Value: The RFC authors suggest code point 0x1A (26 decimal) as a suitable unassigned value. The final value will be assigned by IANA upon publication; the value 0xC8 used throughout examples in this document is a non-normative placeholder. Implementations **MUST NOT** hardcode 0xC8 and **SHOULD** parameterize the block-type code so that the IANA-assigned value can be substituted post-publication.

This document additionally requests IANA to create a "CPB Metric Types" sub-registry of the Bundle Block Types registry. The registration policy for the sub-registry is Specification Required. Initial registrations are:

- 0: PRoPHET Delivery Predictability
- 1: CGR Contact Confidence
- 2: MaxProp Path Cost (inverted)
- 3: RAPID Utility Metric
- 4: Generic Probability

Values 5-255 are unassigned. Future assignments in this range require a specification and expert review.

10. Overhead Analysis

Payload (bytes)	Base Bundle	CPB minimal (8 bytes)	CPB full (32 bytes)	Overhead Range
50	~100	~108	~132	8-32%
256	~306	~314	~338	2.6-10.5%
1024	~1074	~1082	~1106	0.7-3.0%
4096	~4146	~4154	~4178	0.2-0.8%

Table 2: CPB Byte Overhead vs Payload Size

For kilobyte-scale payloads typical of DTN sensor data, overhead is below 3% even with full CPB fields.

Empirical measurement on the reference encoder in the 2-node ION campaign under `impl/real-cpb-ion-test/` (see also Section 12.6) produced 51 bytes of overhead for a full-field CPB (float16 default probability + path entries + timestamp + validity + metric type + confidence) plus its BTSD wrapper when added to a minimal BPv7 bundle (76 B plain → 127 B with CPB). That full-field figure is distinct from the minimal-map timeline1 examples in Section 12.6 (e.g., a 121 B with-CPB sample carrying only default probability and one path entry). Full-field overhead is higher than the idealized "CPB full (32 bytes)" row above because of concrete CBOR map/array encoding and the BTSD block structure; it remains small in absolute terms for any realistic DTN ADU.

10.1. Computational and Storage Overhead

The table above quantifies only wire-byte overhead. CPB consumption imposes additional costs on forwarders that **SHOULD** be considered when sizing a CPB-enabled deployment, particularly on constrained nodes:

CBOR decode cost
: Decoding the CPB map (up to 8 path entries) requires CBOR parsing of approximately 32-100 bytes per CPB. On commodity hardware this is sub-millisecond; on constrained microcontrollers with software floating-point it may be several milliseconds per CPB.

BIB verification cost
: If the CPB is BPSec-protected, each consumption requires a signature verification. Public key verification (ECDSA, Ed25519, HMAC) costs vary by algorithm and hardware support. In the worst case (resource-constrained node, no hardware crypto), BIB verification can dominate CPB processing time. Implementations **SHOULD** cache verification results per (block-number, BIB-block-number) tuple within a single bundle's processing window.

Route recomputation
: Each new CPB observation potentially triggers route recomputation. For a confidence-weighted routing policy consuming CPB data (see CGR-UCoP [Fraire18] as prior art), the cost is dominated by the contact-graph traversal, which scales as O(C log C) where C is the contact-plan size; CPB consumption adds a per-edge multiplication. The hysteresis and rate-limiting techniques of Section 4.4 bound the recomputation rate.

Memory pressure from multi-CPB accumulation
: If the strip-and-replace model (Section 8.3.2) is not used, bundles can accumulate multiple CPBs across hops. Each CPB occupies up to ~32 bytes plus BIB overhead. Bundle store memory pressure **SHOULD** be considered when designing CPB production policy; the optional optimization in Section 3.6 (a node appending a new CPB **MAY** remove earlier CPB instances) is relevant here.

Aging-logic cost
: Maintaining timestamp-aware probability decay (Section 4.1) requires periodic re-evaluation of cached CPB values against the current clock. This is O(1) per cached CPB but the cache itself **SHOULD** have a bounded size.

On constrained DTN nodes, CPU and storage costs frequently exceed wire-byte costs in their operational impact. Concrete profiling numbers depend on the BP implementation, the cryptographic algorithms in use, and the deployment topology, and are not provided in this document. Implementations **SHOULD** publish their measured overhead alongside interoperability reports per [RFC7942].

11. Implementation Status

This section is to be removed before publishing as an RFC.

[Note to RFC Editor: per [RFC7942], this section may be removed before publication.]

This section records the status of implementations of the Contact Probability Block at the time of writing, in accordance with [RFC7942].

Reference Python Implementation

- **Organization:** Independent (N. Perry)
- **Repository:** https://github.com/luckyseoul/draft-perry-dtn-cpb
- **License:** MIT License
- **Modules:** cpb.py (encoder/decoder), test_cpb.py (conformance suite for wire encode/decode), config1_sim.py (discrete-event simulator producing the Section 12 Configuration 1 results).
- **Coverage:** Wire-format encode and decode of CPB block-type-specific data (CBOR map fields, float16 encoding, invalid-float handling, BTSD wrapping), byte-exact match against Figure 2 and Figure 7 and the hex table in Section 3.4.3, and acceptance of more than eight path entries consistent with the SHOULD (not MUST) path-array size guidance. Routing policies, multi-CPB precedence, per-path matching algorithms, BPSec flags/CRC handling, and intermediate-router update rules are specified in this document but are not automated unit tests in the reference encoder.
- **Maturity:** Reference implementation. Conformance tests pass on Python 3.10+ with cbor2 5.9.0. Round-trip byte stability verified across multiple encode/decode cycles.
- **Contact:** nick@perrybrandsllc.com

Simulation-Based Validation Validation of CPB's routing impact was conducted via the pure-Python discrete-event simulator config1_sim.py, which implements vanilla CGR (earliest-arrival) and the Section 12 consumer labeled "cpb" (confidence-weighted cost) over Configuration 1 (4 rovers, 4 orbiters, 1 relay, 3 ground stations). The simulator consumes ground-truth confidences of the class CPB carries; it does not encode or decode on-wire CPB extension blocks. Section 12 reports the authoritative 10-seed paper-battery results for those two policies, reproduced by `python3 config1_sim.py --battery paper --strategy both`. The simulator is fully deterministic per seed on Python 3.10+ (contact CRN does not require cbor2; the encoder does). CGR-UCoP [Fraire18] was used only as prior art for simulation parameters and basic validation.

PWG ION Data-Plane Validation Limited real end-to-end data-plane results on NASA ION v4.1.0 (PWG Tailscale mesh) are reported in Section 12.6: creation and injection of bundles carrying experimental extension block type 200, UDPCL carriage, bplist visibility, and byte-exact CPB CBOR survival. Artifacts live under `impl/real-pwg-deployment/timeline1/` and `impl/real-cpb-ion-test/`. This is not an in-band CPB-consuming routing experiment on ION CGR.

Implementations Considered but Not Exercised The author also installed and partially configured The ONE Simulator during development of this specification:

- **The ONE Simulator v1.6.0:** Installed and a probability-aware Spray-and-Wait router was drafted. No results from The ONE are reported in Section 12.

No other implementations are known to the author at the time of writing. Reports of new implementations are encouraged.

12. Experiment Description

This section describes the experiment by which the Contact Probability Block specification is validated. Per [RFC2026] Section 4.2.1, an Experimental specification is accompanied by a description of the experiment in concrete, falsifiable terms.

12.1. Scope and Hypothesis

The experiment has two complementary parts that together support the CPB design goal (a BPv7 extension block that carries per-contact probability metadata for routing consumers):

1. **Wire-format correctness.** The reference encoder produces CPB block-type-specific data per Section 3, and the live ION data-plane runs in Section 12.6 show that experimental extension block type 200 carrying that data survives creation, injection, and UDPCL carriage on a real BPv7 stack.
2. **Routing value of the metadata class.** Configuration 1 compares confidence-blind earliest-arrival CGR to a confidence-weighted consumer of ground-truth per-contact confidences of the same class CPB is designed to carry (metric-type 1). The discrete-event simulator does not place extension blocks on a simulated wire; repository bridge tests verify that Configuration 1 confidences encode and decode as CPB maps without changing the confidence-weighted cost ranking.

The core hypothesis for part (2) is that such confidence metadata, when available to routing, can improve delivery and latency under stochastic contact failures with multiple competing routes. In-band consumption of CPB by production CGR (or other stacks) is not claimed here and is listed under Section 12.11.

Delivery ratio, mean latency, and p95 latency are all reported for Configuration 1. Delivery ratio is the primary success criterion for the routing-value hypothesis; mean latency is a secondary operational metric. p95 is reported for completeness and is not assumed to move with the mean (see Section 12.5).

12.2. Test Configuration 1: Multi-Asset Mars Relay

The experiment uses a multi-asset Mars-to-Earth relay topology with four surface assets, four orbiters, one deep-space relay, and three ground stations. All node identifiers in the topology use the "ipn" URI scheme as updated by [RFC9758]; node numbers shown are Default Allocator FQNNs.

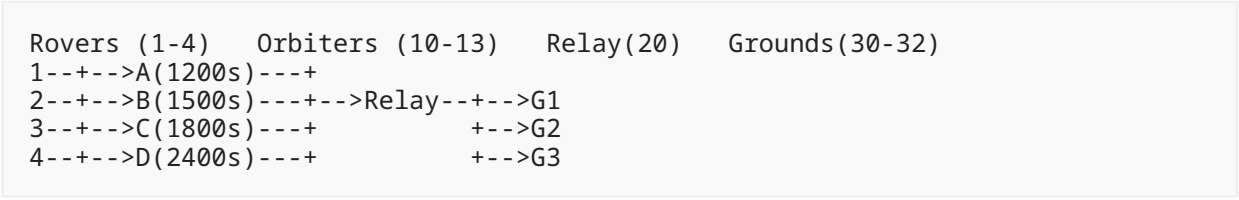

Figure 8: Test Topology

Rover	Orbiter-A (10)	Orbiter-B (11)	Orbiter-C (12)	Orbiter-D (13)
1	0.78	0.92	0.85	0.96
2	0.85	0.78	0.96	0.92
3	0.92	0.96	0.78	0.85
4	0.96	0.85	0.92	0.78

Table 3: Per-(Rover, Orbiter) Success Probabilities

12.3. Routing Policies Tested

CPB is a BPv7 extension block that carries per-contact probability metadata. The policies below are simulator-internal consumers of that metadata. CGR-UCoP [Fraire18] is referenced only as prior art consulted during construction of the simulation parameters and for basic cross-validation; it is not the conceptual basis for the CPB block itself.

Baseline: Vanilla CGR (earliest arrival) Standard Contact Graph Routing. The cost of a candidate route is its earliest predicted arrival time. Confidence values are ignored.

Confidence-weighted CPB consumer (labeled "cpb") Experimental consumer of ground-truth confidences of the class carried by CPB metric-type 1. Route cost is `latency / confidence`, where confidence is the end-to-end path success-probability product. Bottleneck data rates are modeled in the topology for operational realism but are not folded into this published cost, so observed gains are not confounded with pure rate-only routing. Other consumers (including rate-aware variants) are possible.

12.4. Methodology

The published simulator is config1_sim.py (Configuration 1): 4 rovers, 4 orbiters, 1 deep-space relay, and 3 ground stations. Rover-orbiter confidences are in [0.78, 0.96]; space-side hops use 0.99. Per-hop data rates are present in the topology with a latin-square assignment that is not rank-aligned with the confidence table (separable factors). The published "cpb" cost uses confidence only. Each hop allows up to three contact-window trials (failed windows advance to the next cycle). Each run covers 7 days of simulated time with bundles generated every 30 s per rover after warm-up (≈80k bundles per arm per seed).

The paper battery uses 10 independent seeds. For each seed and each policy, the simulator uses the same seed and strategy-independent common-random-number (CRN) contact trials so that the same contact window and trial index yields the same Bernoulli outcome regardless of which route was chosen. The policies exercised (and shipped in config1_sim.py) are:

- Baseline (vanilla CGR): earliest-arrival routing with no use of per-contact confidence.
- Confidence-weighted CPB consumer ("cpb"): cost = latency / confidence.

12.5. Results

Results below are for Configuration 1 as implemented by config1_sim.py, 10 independent seeds, paper battery (`--battery paper --strategy both`).

For each seed, baseline and "cpb" were exercised with shared CRN contact outcomes (same seed, contact id, and trial index).

Primary findings (Configuration 1, 10 seeds, paper battery via config1_sim.py --battery paper --strategy both):

- Delivery ratio: baseline mean delivery was 0.9965; "cpb" reached mean delivery 0.9984. Paired (cpb − baseline) delivery mean gain +0.00191 (t ≈ 7.0 over 10 seeds).
- Mean latency: baseline mean lat_avg was ≈508 s; "cpb" reduced mean lat_avg to ≈484 s (paired mean change ≈ −24 s, t ≈ −9.7).
- Tail latency: baseline mean p95 was ≈1604 s; "cpb" mean p95 was ≈1665 s (paired mean change ≈ +60 s). On this topology confidence-weighting does **not** improve p95; the experiment does not claim a universal tail win.
- Path confidence: mean chosen path product was higher under "cpb" (0.9016) than baseline (0.8600).

Interpretation. On Configuration 1, the confidence-weighted consumer of CPB-class ground-truth confidences improves delivery ratio and mean latency relative to confidence-blind earliest-arrival CGR, while selecting higher path confidence. p95 latency is topology- and cost-dependent and was worse under "cpb" here; operators caring about tails must measure their own cost functions. Other consumers of CPB metadata are possible; this experiment evaluates one concrete cost against the baseline under CRN contact trials.

Limitations and caveats. Results are for the Configuration 1 topology and cost formula as shipped in config1_sim.py. The study does not address noisy or adversarially supplied confidence values; those questions are left for future work.

12.6. Real-Stack Data-Plane Validation (ION PWG Testbed)

In addition to the discrete-event simulation, a limited real data-plane validation was executed on a three-node NASA ION v4.1.0 testbed on the IPNSIG PWG Tailscale mesh, plus dedicated emulated targets. The real nodes were ipn node numbers 268485122, 268485121, and 268485123. Emulated Moon (268484801), Mars (268484820), and gateway (268485000) endpoints were reached via contact plans to a public gateway on the mesh. All real nodes used UDPCL on port 4556 with long-duration bidirectional contacts, CFDP and bputa for CPB injection (CFDP entities configured for the emulated EIDs), ionsec default key, and DTNEx for dynamic CBOR contact and probability exchange on service numbers .12160/.12161. Host firewalls allowed UDP 4556 (UDPCL), 4557, and 12160 (DTNEx) from the Tailscale address ranges used by the mesh.

Two phases were executed using the reference packet.py and cpb.py tools (minimal CPB map with default probability and path entries encoded as IEEE binary16 floats per Section 3.4; the deployed encoder for these runs did not insert a format-version key).

Phase 1 (72 messages): Each of the three real nodes sent eight fixed short text payloads with a CPB (using eight fixed test probabilities in 0.65..0.99) to each of three dedicated test EIDs (gateway 268485000, Moon 268484801, Mars 268484820). Bundles were created with source equal to the sending real EID and injected via bputa to the destination CFDP entity after cfdpadmin configuration (bputa enabled, discard=0, segsize=1400, requirecrc=0). cfdpclock and bputa were confirmed running on the injector before each batch.

Phase 2 (32 messages, emulated test nodes only): The same eight payloads were sent bidirectionally (Moon→Mars and Mars→Moon), both with a CPB and plain (no CPB block). Bundles were created with source equal to the originating emulated EID and destination equal to the other emulated EID, then injected via bputa from node 268485122 to the destination CFDP entity (268484820 for Moon→Mars; 268484801 for Mars→Moon). This batch was limited to the two emulated test nodes.

All bundles (104 total) were produced by the reference encoder, successfully injected, and appeared in bplist for the target EIDs (often queued for forwarding). Phase 2 with-CPB examples use a minimal CPB map (default probability plus one path entry); a representative file is 121 B (see dissection below). The separate full-field overhead measurement of 51 B (76 B plain → 127 B with CPB) is from the 2-node campaign in `impl/real-cpb-ion-test/` and is reported under Section 10; it is not the size of these minimal-map bundles. Artifacts are preserved under `impl/real-pwg-deployment/timeline1/`.

Wire-format evidence: Wireshark 4.6.4 (with tshark) on node 268485122 confirmed BPv7 and BPv7 Block Type dissectors. A manual CBOR and hex dissection was performed on moon_to_mars_with_cpb_01_save.bundle (121 B; first fixed payload with probability approximately 0.65). The full wire bytes are:

```
88070000498201821a1000c0c101498201821a1000c0d401498201821a1000c0
c101821a6a23cf0800190e10830600498201821a1000c0c101830a0082001820
8518c80011005150a200f939330181821a1000c0d4f9393384010000581b7361
7665206120686f7273652c207269646520612063 6f77626f79
```

Primary block (first CBOR array, 40 bytes): version 7, flags 0, CRC type 0, source ipn:268484801.1, destination ipn:268484820.1, report-to same as source, creation timestamp [1780731656, 0], lifetime 3600. This matches the primary constructed by packet.py:create_bundle_with_cpb.

Previous Node block (type 6): [6, 0, source EID].

Hop Count block (type 10): [10, 0, [0, 32]].

Extension block 200 (the CPB, bytes starting at offset 0x40): [200, 0, 0x11 (REPLICATE|DISCARD), 0, BTSD]. The BTSD bstr is 50a200f939330181821a1000c0d4f93933, which decodes to the canonical CBOR map {0: 0.64990234375 (float16 0xf93933 per draft 3.4), 1: [[268484820, 0.64990234375]]}. This is byte-exact to the output of cpb.encode_btsd() for the dict passed from create_bundle_with_cpb (F_DEFAULT_PROB and single F_PATH_ENTRIES entry; no F_VERSION key in the runtime encoder used for these runs).

Payload block (type 1): a fixed 27-byte ASCII test string used for this campaign.

The CPB block and its single direct path entry matched the encoder output bit-for-bit; no appended path entries or probability mutations were present. The same structure (varying only the probability value and target node in the path entry) appears in all 16 with-CPB bundles from the Phase 2 batch.

Receive-side collection: bpsink, a bprecv-based receiver, and recv_cpb.py were run on the emulated EID services (.1 and .12160) across the three real nodes for a 30 s window after the Phase 2 injections. Captured logs under timeline1/receive_logs/ showed no local application delivery of the test payloads and no receive-side CPB decodes in that window.

That outcome is consistent with CFDP-wrapped bputa injection and with emulated targets that are plan and CFDP-entity constructs only (bundles often remained in bplist forwarding queues rather than delivering to a .1 endpoint).

Decode verification (post-injection): recv_cpb.py and the receiver scanner were run against the source .bundle files and captures. Every with-CPB bundle decoded to its original map (single direct path entry and the assigned test confidence). No path updates or probability changes were observed on those files.

Artifacts (Phase 1 and Phase 2 bundles, receiver logs, bplist snapshots, host configuration files, and run notes) are under `impl/real-pwg-deployment/timeline1/`. Related two-node full-field samples are under `impl/real-cpb-ion-test/`.

Limitations of this validation (distinct from the simulation). The emulated targets (268484801 and 268484820) are plan and CFDP-entity constructs only; full ION daemons do not run on those EIDs. Many bundles therefore remained queued for forwarding rather than being delivered as application data. Phase 2 injection used bputa on node 268485122 toward the destination CFDP entity (a short span within the plan/CFDP layer on one host). Full in-band consumption of CPB values by ION CGR or DTNEx for forwarding decisions was not exercised. Injection of pre-formed bundles required bputa; standard bpsource does not provide an equivalent path for embedding an experimental extension block. No BPSec was applied in these runs. The validation demonstrates creation, bputa/CFDP injection, UDPCL carriage over a multi-node mesh with DTNEx, bplist visibility, and wire-format survival of experimental extension block type 200 carrying the CPB CBOR map as encoded. It does not constitute a multi-hop routing-policy experiment with in-band CPB consumption on production stacks.

12.7. Goals (per RFC 2026 Section 4.2.1)

1. Determine whether a confidence-weighted consumer of CPB-class ground-truth confidences improves routing metrics versus confidence-blind earliest-arrival CGR under stochastic contact failures on Configuration 1. **Met for delivery and mean latency** (delivery 0.9965→0.9984, lat_avg ≈508→484 s); p95 not improved (see Section 12.5).
2. Quantify CPB byte overhead. **Met by Section 10 analytical bounds:** <3% for 1 KB+ payloads with full CPB.
3. Demonstrate reference encoder self-consistency (byte-stable round-trips against the figures and hex table). **Met:** conformance suite passes on Python 3.10+ with cbor2.
4. Assess float16 precision sufficiency. **Met:** binary16 quantization produces identical route selections under the tested policies in the configurations used.

12.8. Success Criteria (Met)

1. Statistically significant improvement ($p < 0.05$) on the primary metric. **Met** for delivery: paired (cpb − baseline) mean gain +0.00191 with $t \approx 7.0$ (10 seeds). Mean latency also improved ($t \approx -9.7$); p95 did not.
2. Sample size sufficient for the claims made. **Met:** 10 independent seeds on Configuration 1, with each policy exercised under shared CRN contact trials per seed (≈80k bundles per arm per seed).
3. CPB byte overhead below 5% for 1 KB+ payloads. **Met analytically.**
4. No fundamental architectural conflicts with BPv7. **Met technically:** CPB is a standard extension block, EID immutability is preserved, BPSec integration is straightforward.

Operational conflicts (trust framework, multi-domain CPB interaction, control-theoretic stability) are addressed at specification level but not closed by experimental demonstration.

12.9. Failure Criteria (Not Triggered)

1. No measurable improvement. **Not triggered.**
2. CPB byte overhead exceeds 10% for 1 KB payloads. **Not triggered.**
3. CBOR encoding incompatibilities. **Not triggered.**
4. BPSec integration produces irrecoverable failures. **Not exercised in this configuration** (deferred to follow-up work).

12.10. Deferred Configurations

The following configurations are intentionally out of scope for this Experimental revision and are deferred to follow-on work:

- Spray-and-Wait (or other replication protocols) in dense multi-neighbor mobile topologies where concurrent encounters differentiate policies from earliest-arrival CGR.
- Multi-hop in-band CPB create/update/strip chains with BPSec BIB coverage on a production stack (ION, HDTN, or similar).
- In-band consumption of CPB fields by ION CGR (or DTNEx) for live forwarding decisions.
- Multi-CPB accumulation and precedence across long paths (tens of hops) under partial deployment.

The reference encoder verifies single-block encode/decode stability (including multi-field maps and path arrays). It does not by itself demonstrate multi-hop accumulation of distinct CPB blocks along a path.

12.11. Open Experimental Questions

Results so far support two limited conclusions: (1) the wire format is implementable and can survive on a real BPv7 data plane, and (2) confidence-weighted routing fed by accurate confidences of the CPB class outperforms confidence-blind routing on the same contact-failure realizations in Configuration 1. The items below remain open; this document does not close them.

Robustness to noisy probability estimation
: The simulator uses ground-truth per-contact probabilities. Real deployments must estimate probabilities from historical observations or model predictions, both of which are noisy. Sensitivity analysis under varying estimator noise (e.g., additive Gaussian noise on confidences, biased estimators, estimator drift over time) is needed to characterize the operational regime where CPB-driven routing remains beneficial.

Behavior under stale probabilities
: The age-threshold mechanism (Section 4.1) defines when a CPB is discarded, but routing behavior *just before* the threshold fires (when probabilities are old but not yet invalid) is unstudied. A controlled experiment varying CPB staleness against routing outcome would inform the default max_age values.

Adversarial probability injection The trust model (Section 8.2) provides structural defenses, but their empirical effectiveness against specific attack patterns (sinkhole inflation, blackhole deflation, oscillation-induction, herding-induction) has not been measured. A red-team experiment with one or more adversarial CPB producers in the network would quantify the cost of attack vs. the cost of defense.

Conflicting CPBs from multiple administrative domains Section 8.2.5 defines policy options but does not demonstrate which option works in practice. A multi-domain testbed with explicit trust boundaries would characterize the gateway-translation, accept-all, and discard-all strategies under realistic traffic.

Partial deployment behavior CPB is designed for incremental deployment: nodes that don't understand CPB pass it through unchanged. But the routing *quality* in a mixed CPB-aware / CPB-unaware network is not characterized. Sensitivity to the fraction of CPB-aware nodes would inform deployment phasing.

Congestion collapse and storage pressure Probability-aware routing concentrates traffic on high-confidence routes (Section 4.4). Under sustained load this risks congestion collapse on the preferred routes and storage pressure at intermediate nodes. The mitigations of Section 4.4 are **RECOMMENDED** but unmeasured. A high-load experiment varying bundle generation rate against delivery ratio would test the failure mode directly.

Custody transfer interactions CPB has been designed not to interfere with BP custody transfer mechanisms ([RFC9171]), but the combined behavior of custody-bound bundles carrying CPB data has not been exercised end-to-end. Particular attention should be paid to whether custody acknowledgments themselves carry CPB-relevant information and whether custody transfer policies should consume CPB.

Heterogeneous fragmentation behavior Section 3.2.1 defines CPB behavior under fragmentation, but only the reference Python encoder has been exercised against it. Real BP implementations (ION, HDTN, uPCN) have subtly different fragmentation pathways and CPB replication behavior across implementations is unproven.

Interoperability with production BP stacks Interoperation between at least two independent implementations on real BPv7 stacks (for example NASA ION and an independently developed alternative such as HDTN, µPCN, or a fresh implementation) has not been demonstrated. The reference Python implementation (Section 13) was developed with this specification by the same author and is not a second independent stack. Partial data-plane evidence on a NASA ION v4.1.0 three-node PWG mesh plus emulated targets is described in Section 12.6: extension blocks were injected via bputa, carried over UDPCL with DTNEx active, and verified intact on the wire. Full consumption of CPB values by ION CGR or DTNEx for forwarding decisions, multi-hop in-band updates, and BPSec-protected bundles remain open.

Control-theoretic stability analysis Section 4.4 identifies feedback dynamics qualitatively and recommends mitigations, but does not provide a formal stability proof. Analyzing the routing system as a discrete-time dynamical system with CPB-driven feedback remains future work for dense networks.

Metric-type cross-validation Section 3.5 prohibits cross-metric arithmetic but does not validate the prohibition empirically. Whether a router making routing decisions from mixed-metric CPBs in practice produces worse outcomes than a router strictly partitioning by metric-type is an open empirical question.

The reference implementation and simulator (Section 13) may assist follow-on measurement of any of the items above.

13. Reference Implementation

The reference implementation supporting this specification consists of the following Python modules under `impl/`:

- **cpb.py** - CBOR encoder/decoder, implementing the CDDL schema in Section 3.2 and the encoding rules in Section 3.4.
- **test_cpb.py** - wire-format conformance suite (encode/decode, hex tables, invalid floats, path-array SHOULD). Passes on Python 3.10+ with cbor2 5.9.0.
- **config1_sim.py** - the discrete-event simulator that produced the Configuration 1 results in Section 12 (paper battery).
- **test_config1_policies.py** - unit tests for the confidence-weighted cost formula.
- **test_sim_cpb_bridge.py** - bridge tests showing Configuration 1 confidences encode/decode as metric-type 1 CPB maps without changing confidence-weighted route cost ranking.
- **test_draft_consistency.py** - structural checks that draft and documentation claims stay aligned with shipped code.

13.1. Repository

Source code is published under the MIT License at:

https://github.com/luckyseoul/draft-perry-dtn-cpb

13.2. Reproducing the Experiment Results

```
<CODE BEGINS>
git clone https://github.com/luckyseoul/draft-perry-dtn-cpb
cd draft-perry-dtn-cpb
pip install cbor2

python3 impl/test_cpb.py
python3 impl/test_config1_policies.py
python3 impl/test_sim_cpb_bridge.py
python3 impl/test_draft_consistency.py
python3 impl/config1_sim.py --battery paper --strategy both

<CODE ENDS>
```

Figure 9: Reproduction Commands

The paper battery run takes on the order of tens of seconds and should reproduce the Section 12.5 mean delivery figures for the published R=3 methodology (baseline about 0.9965, cpb about 0.9984).

The 2-node real-stack transmission results (with-CPB vs plain bundles using the 8 fixed test confidence values, bputa injection after cfdpadmin rc update, and observed delivery) are reproduced from the exact bundles and instructions in impl/real-cpb-ion-test/ (see real_cpb_ion_results.md, bundles/*.bundle and *.b64, the updated .rc files in config/, packet.py for bundle assembly, and recv_cpb.py for post-receipt CPB decoding on a

captured bundle file). Full end-to-end transmission requires the specific ION testbed hardware, Tailscale link, and IPN activation state; the bundle creation, cfdp rc changes, bputa invocation, and decode steps are self-contained and runnable on any ION installation that supports bputa.

13.3. Relationship to Updates of BPv7

[RFC9171] (BPv7) has been updated by [RFC9713] (administrative record types registry, January 2025) and [RFC9758] (ipn URI scheme updates, May 2025) since this CPB work began. Neither update affects the wire format defined in Section 3:

- [RFC9713] concerns administrative record types; CPB is an extension block, not an administrative record, so RFC 9713 does not apply directly.
- [RFC9758] updates the "ipn" URI scheme. CPB implementations **MUST** encode and decode ipn EIDs per RFC 9758, including Default Allocator FQNNs. The LocalNode ipn URI form (Allocator=0, NodeNumber=2^32 - 1) has no meaning off-node and **MUST NOT** appear in CPB path-entries transmitted to other nodes.
- RFC 9171 erratum 8376 notes that BPv7 ADU fragmentation interacts ambiguously with extension blocks. CPB's fragmentation handling is defined in Section 3.2.1.

14. Normative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/info/rfc8174>.

[RFC9171] Burleigh, S., Fall, K., and E. Birrane, "Bundle Protocol Version 7", RFC 9171, DOI 10.17487/RFC9171, January 2022, <https://www.rfc-editor.org/info/rfc9171>.

[RFC9172] Birrane, E. and K. McKeever, "Bundle Protocol Security (BPSec)", RFC 9172, DOI 10.17487/RFC9172, January 2022, <https://www.rfc-editor.org/info/rfc9172>.

[RFC8949] Bormann, C. and P. Hoffman, "Concise Binary Object Representation (CBOR)", STD 94, RFC 8949, DOI 10.17487/RFC8949, December 2020, <https://www.rfc-editor.org/info/rfc8949>.

[RFC8610] Birkholz, H., Vigano, C., and C. Bormann, "Concise Data Definition Language (CDDL): A Notational Convention to Express Concise Binary Object Representation (CBOR) and JSON Data Structures", RFC 8610, DOI 10.17487/RFC8610, June 2019, <https://www.rfc-editor.org/info/rfc8610>.

[RFC8126] Cotton, M., Leiba, B., and T. Narten, "Guidelines for Writing an IANA Considerations Section in RFCs", BCP 26, RFC 8126, DOI 10.17487/RFC8126, June 2017, <https://www.rfc-editor.org/info/rfc8126>.

[RFC2026] Bradner, S., "The Internet Standards Process -- Revision 3", BCP 9, RFC 2026, DOI 10.17487/RFC2026, October 1996, <https://www.rfc-editor.org/info/rfc2026>.

[RFC9713] Sipos, B., "Bundle Protocol Version 7 Administrative Record Types Registry", RFC 9713, DOI 10.17487/RFC9713, January 2025, <https://www.rfc-editor.org/info/rfc9713>.

[RFC9758] Taylor, R. and E. Birrane, "Updates to the "ipn" URI Scheme", RFC 9758, DOI 10.17487/RFC9758, May 2025, <https://www.rfc-editor.org/info/rfc9758>.

15. Informative References

[IPNSIG2023] Kaneko, Y., Cerf, V., Tabunshchyk, H., Burleigh, S., Blanchet, M., Scott, K., Suzuki, K., Garcia, O., Chappell, L., Danielson, H., Pace, S., Green, J., and J. Schier, "Solar System Internet Architecture and Governance -- from the Moon to Mars and beyond", September 2023, <https://www.ipnsig.org/_files/ugd/d716c2_5df0d50ff8ab466380dadc75e166cfcf.pdf>.

[RFC6693] Lindgren, A., Doria, A., Davies, E., and S. Grasic, "Probabilistic Routing Protocol for Intermittently Connected Networks", RFC 6693, DOI 10.17487/RFC6693, August 2012, <https://www.rfc-editor.org/info/rfc6693>.

[RFC9675] Birrane, E., Heiner, S., and E. Annis, "Delay-Tolerant Networking Management Architecture", RFC 9675, DOI 10.17487/RFC9675, October 2024, <https://www.rfc-editor.org/info/rfc9675>.

[RFC7942] Sheffer, Y. and A. Farrel, "Improving Awareness of Running Code: The Implementation Status Section", BCP 205, RFC 7942, DOI 10.17487/RFC7942, July 2016, <https://www.rfc-editor.org/info/rfc7942>.

[Fraire18] Fraire, J., Madoery, P., Burleigh, S., Feldmann, M., Finochietto, J., Charif, A., Zergainoh, N., and R. Velazco, "Assessing Contact Graph Routing Performance and Reliability in Distributed Satellite Constellations", IEEE Transactions on Aerospace and Electronic Systems, DOI 10.1109/TAES.2017.2738278, 2018, <https://doi.org/10.1109/TAES.2017.2738278>.

[Spyropoulos05] Spyropoulos, T., Psounis, K., and C. S. Raghavendra, "Spray and Wait: An Efficient Routing Scheme for Intermittently Connected Mobile Networks", WDTN '05: Proceedings of the 2005 ACM SIGCOMM workshop on Delay-tolerant networking, DOI 10.1145/1080139.1080143, 2005, <https://doi.org/10.1145/1080139.1080143>.

[CGREB] Birrane, E., "Contact Graph Routing Extension Block (CGREB) (expired)", Work in Progress, Internet-Draft, draft-irtf-dtnrg-cgreb-00, 2013, <https://datatracker.ietf.org/doc/html/draft-irtf-dtnrg-cgreb-00>.

[SABR] CCSDS, CCSDS., "Schedule-Aware Bundle Routing", CCSDS 734.3-B-1, July 2019, <https://public.ccsds.org/Pubs/734x3b1.pdf>.

[CGR] Burleigh, S., "Contact Graph Routing in Delay Tolerant Networks", 2010, <https://doi.org/10.1109/MAES.2010.5546306>.

Appendix A. Mars Mission Scenario

Consider a Mars rover (node 200) communicating with an Earth relay station via an orbiter (node 100):

1. **Initial contact:** Rover creates a critical status bundle with a CPB indicating default probability 0.6 (dusty conditions), metric-type 1 (cgr-confidence), validity_duration 3600s
2. **Rover-to-orbiter:** Router at rover checks 0.6 < 0.7 -> wait for better conditions
3. **Later contact:** Dust clears, CPB is refreshed with probability 0.95, new timestamp
4. **Orbiter processes:** Router at orbiter checks 0.95 > 0.7 -> forward immediately
5. **Result:** Both bundles are delivered with minimal latency.

Decay example: If the first bundle (P=0.6) is delayed 10 hours and the validity_duration was 3600s, receiving routers recognize the CPB as expired and apply decay: P_aged = 0.6 * 0.98^10 = 0.49.

Appendix B. Change Log

This section is to be removed before publishing as an RFC.

[RFC Editor Note: Please remove this entire section before publication.]

B.1. draft-perry-dtn-cpb-00 (July 2026)

- Initial Experimental -00: Contact Probability Block wire format, metric-type registry, operational and security considerations, and IANA requests.
- CBOR float16 encoding with deterministic encoding rules; format version field (key 7); per-path array size guidance as SHOULD with local enforcement on constrained links.
- BPSec integrity models (append-only and strip-and-replace); BCB encryption of CPB prohibited for routing use.
- Relationship to PRoPHET, MaxProp, RAPID, Spray-and-Wait, CGR, and DTNMA; IPNSIG Section 4.2 problem framing.
- Configuration 1 simulation (baseline earliest-arrival CGR vs one confidence-weighted consumer) with CRN contact trials; delivery and mean latency improvements reported; p95 not improved on that topology.
- Limited ION data-plane validation of extension-block carriage (not in-band CGR consumption of CPB).
- Reference implementation and reproduction commands (Section 13).
- Aligns with RFC 9171 errata relevant to CRC handling and fragmentation; cites [RFC9713] and [RFC9758] as updates related to BPv7 administration and the ipn scheme.
- CDDL, BTSD embedding, fragmentation pruning, timestamp tie-breaker, and IANA text.

Author's Address

Nicholas Perry
Independent
Council Bluffs, Iowa
United States of America
Email: nick@perrybrandsllc.com